SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I Shaik Mohammed Waseem Rayan(192373004) certify that this submission is my original work and that I have adhered
to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Shaik Mohammed Waseem Rayan

Annexure A

Reverse Engineering

1.Machine Learning Techniques for Reverse Engineering

Description:

Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. Students study supervised, unsupervised, and semi-supervised learning approaches used in reverse engineering.

Student Activity:

- Collect a dataset of source-code samples.
- Extract code features.
- Apply classification or clustering algorithms.
- Analyze similarities among programs.
- Compare the performance of different ML algorithms.

Expected Outcome:

Students will evaluate the suitability of different ML algorithms for software reverse-engineering tasks.

2. AI for Source Code Analysis and Program Understanding

Description:

AI can analyze large source-code repositories and automatically identify functions, classes, variables, dependencies, and programming patterns. Natural Language Processing and Large Language Models can also convert complex code into human-readable explanations.

Student Activity:

- Select a source-code repository.
- Analyze the code using AI-assisted tools.
- Identify important functions and dependencies.
- Generate natural-language explanations.
- Verify the AI-generated explanations against the actual code.

Expected Outcome:

Students assess the effectiveness and limitations of AI in understanding unfamiliar source code.

3. AI-Based Malware Reverse Engineering and Analysis

Description:

AI can assist security researchers in analyzing malware behavior by identifying suspicious code patterns, API calls, strings, functions, and execution behaviors. Students can study the conceptual workflow of AI-assisted malware analysis using **safe, non-executable datasets or publicly available research samples**.

Student Activity:

- Study malware-analysis datasets.
- Identify relevant static features.
- Apply ML classification techniques.
- Analyze suspicious patterns.
- Evaluate classification accuracy.

Expected Outcome:

Students understand and evaluate how AI can support automated malware analysis.

4. Reverse Engineering of APIs Using AI

Description:

APIs may need to be understood when documentation is incomplete or unavailable. AI can analyze API requests, responses, source code, and observed behavior to infer endpoints, parameters, data formats, and relationships.

Student Activity:

1. Select a documented/open API or controlled test API.
2. Analyze its request-response structure.
3. Use AI to infer API functionality.
4. Generate documentation.
5. Compare generated documentation with the actual API specification.

Expected Outcome:
Students assess the usefulness of AI for API understanding and documentation recovery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

1. Machine Learning Techniques for Reverse

Engineering Introduction

Reverse engineering is the process of analyzing software to understand its structure, functionality, and behavior without relying entirely on its original documentation. Machine Learning (ML) can improve reverse engineering by automatically identifying programming patterns, classifying software components, detecting anomalies, and measuring similarities between programs.

The main challenge is that large software systems contain thousands of functions, variables, dependencies, and code fragments. Manually analyzing all these components is time-
consuming. Machine Learning provides an automated approach for extracting meaningful patterns from source-code datasets.

Problem Identification

The problem is to determine whether Machine Learning algorithms can effectively classify and analyze unknown software components based on their source-code characteristics. The system should extract relevant features from programs, apply ML algorithms, identify similarities or differences, and compare the performance of different algorithms.

Dataset and Feature Extraction

A dataset containing source-code samples written in languages such as **C, C++, Java, or Python** can be collected from open-source repositories.

Important features that can be extracted include:

- Number of lines of code
- Number of functions
- Number of variables
- Number of loops and conditional statements
- Cyclomatic complexity
- Frequency of keywords
- Number of comments
- Function-call patterns

- Token frequency
- Abstract Syntax Tree (AST) characteristics

These features convert source code into numerical representations that can be processed by ML algorithms.

Machine Learning Techniques

6. Supervised Learning

In supervised learning, the source-code samples are assigned predefined labels such as:

- Sorting algorithm
- Searching algorithm
- Encryption-related code
- File-handling code
- Network-related code

Algorithms such as **Decision Tree, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN)** can then be trained to classify unknown programs.

7. Unsupervised Learning

Unsupervised learning is useful when labelled data is unavailable. Algorithms such as **K-Means Clustering** and **DBSCAN** can group programs according to their structural or behavioral similarity.

For example, programs implementing similar algorithms may form the same cluster even if their variable names or coding styles are different.

8. Semi-Supervised Learning

Semi-supervised learning combines a small amount of labelled data with a larger amount of unlabelled data. This is useful in reverse engineering because obtaining manually labelled software datasets can be expensive and time-consuming.

Tools Used

- **Python** – implementation of ML algorithms
- **Pandas** – dataset processing
- **NumPy** – numerical operations
- **Scikit-learn** – classification and clustering
- **AST Parser** – source-code structure analysis
- **Matplotlib/Seaborn** – visualization of results

- **GitHub/Open-source repositories** – source-code datasets

Methodology

The proposed workflow is:

Source Code Collection → Preprocessing → Feature Extraction → Dataset Creation → ML Model Training → Classification/Clustering → Similarity Analysis → Performance Evaluation

The source-code samples are first cleaned and converted into measurable features. These features are then supplied to different ML algorithms. Their predictions or clusters are analyzed to determine which algorithm performs best.

Performance Evaluation

For classification algorithms, performance can be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

For clustering algorithms, measures such as the **Silhouette Score** can be used.

A comparative analysis can determine which ML technique provides the most reliable results for a particular reverse-engineering task.

Expected Result

Machine Learning can significantly reduce the manual effort required for software analysis. Supervised learning is suitable when labelled software samples are available, while clustering is useful for discovering unknown relationships between programs.

Conclusion

Machine Learning provides an effective approach for automating several reverse-engineering tasks. Feature extraction combined with classification and clustering can help identify software components, detect similarities, and discover programming patterns. However, ML predictions depend heavily on the quality and representativeness of the dataset. Therefore, ML should be considered as an assisting technology rather than a complete replacement for human reverse engineers.

---

2. AI for Source Code Analysis and Program

Understanding Introduction

Understanding an unfamiliar software project is a major challenge in software engineering and reverse engineering. Large repositories may contain hundreds of files, classes, functions, libraries, and dependencies. Artificial Intelligence (AI), Natural Language Processing (NLP), and Large Language Models (LLMs) can assist developers in analyzing source code and
converting complex programming structures into understandable natural-language explanations.

Problem Identification

The objective is to determine how effectively AI can understand an unfamiliar source-code repository. The analysis should identify important functions, classes, variables, dependencies, and programming patterns and then generate explanations that can be verified against the actual source code.

Source Code Selection

An open-source repository can be selected from platforms such as GitHub. A moderately sized project is preferable because it contains sufficient complexity for meaningful analysis without becoming difficult to evaluate.

The repository can be analyzed based on:

- File structure
- Classes and functions
- Variables
- Function calls
- External libraries
- Dependencies
- Control flow
- Data flow
- Error-handling mechanisms

AI-Assisted Analysis

AI tools can be used to analyze individual files and explain their purpose.

For example, an AI model can analyze a function and generate an explanation such as:

Input → Validation → Processing → Database/API Operation → Output

The AI can also identify relationships between functions and explain how data moves through the application.

Important Code Components

Functions

AI can identify the purpose of individual functions, their inputs, outputs, and dependencies.

Classes

For object-oriented programs, AI can identify classes, attributes, methods, inheritance relationships, and interactions between objects.

Dependencies

AI can identify external libraries and explain why they are used in the application.

Programming Patterns

AI can detect common patterns such as:

- Singleton
- Factory
- MVC
- Authentication mechanisms
- Error handling
- API communication
- Database access

Tools Used

- **Python** – source-code processing
- **AST Parser** – structural analysis
- **Git/GitHub** – repository access
- **LLM-based AI tools** – code explanation
- **VS Code** – source-code inspection
- **Dependency analysis tools** – dependency identification
- **Graph visualization tools** – dependency representation

Methodology

The methodology follows:

Repository Selection → Code Collection → Structural Analysis → Function/Class Identification → Dependency Extraction → AI Explanation → Human Verification → Accuracy Evaluation

First, the repository is examined manually to understand its basic structure. AI is then used to analyze the same source code. The generated explanations are compared against the actual implementation.

Verification of AI Output

AI-generated explanations should not be accepted blindly. Each explanation must be verified against the original code.

For example, if the AI states that a function validates user credentials, the actual function must be inspected to confirm that it really performs authentication or validation.

Errors may occur because AI can:

- Misinterpret complex logic
- Miss hidden dependencies
- Assume functionality that does not exist
- Confuse variable purposes
- Produce incomplete explanations

Therefore, human verification is an important part of AI-assisted reverse engineering.

Expected Result

AI can significantly reduce the time required to understand unfamiliar source-code repositories. It can provide summaries of functions, identify dependencies, explain complex code, and help create documentation.

However, the accuracy of AI-generated explanations depends on code complexity, context availability, and the quality of the AI model.

Conclusion

AI provides a powerful assistant for source-code analysis and program understanding. LLMs can convert complex programming structures into human-readable explanations and help developers quickly understand unfamiliar projects. However, AI-generated results must be verified against the original implementation because AI can produce incorrect or incomplete interpretations. Therefore, the most effective approach is **AI-assisted analysis combined with human validation**.

---

3. AI-Based Malware Reverse Engineering and

Analysis Introduction

Malware reverse engineering involves analyzing malicious software to understand its functionality, behavior, and potential impact. Traditional malware analysis can require significant manual effort because malware may contain obfuscated code, suspicious API calls, encrypted strings, and complex execution behavior.

Artificial Intelligence and Machine Learning can assist security researchers by automatically identifying suspicious patterns and classifying files based on extracted features. For safety,

this study can be performed using **non-executable malware datasets, feature datasets, or publicly available research samples** rather than executing malware.

Problem Identification

The problem is to determine whether AI and Machine Learning can identify potentially malicious software based on static characteristics. The system should extract relevant features, classify samples, identify suspicious patterns, and evaluate the accuracy of the classification model.

Static Features

Static analysis examines a sample without executing it. Useful features include:

- File size
- File type
- Number of sections
- Imported APIs
- Exported functions
- Strings
- Opcode patterns
- Byte sequences
- Entropy
- Header information
- Number of suspicious functions
- Permission-related characteristics

These features can be converted into numerical values and supplied to ML algorithms.

Machine Learning Techniques

Supervised Classification

A dataset containing labelled benign and malicious samples can be used to train classification models.

Suitable algorithms include:

- Random Forest
- Decision Tree
- Support Vector Machine
- Logistic Regression

- K-Nearest Neighbors

The trained model can then classify previously unseen samples based on their extracted features.

Malware Analysis Workflow

Dataset Collection → Safe Feature Extraction → Data Preprocessing → Feature Selection → ML Training → Classification → Performance Evaluation

Feature selection is particularly important because unnecessary features can increase computational complexity and reduce model performance.

Tools Used

- **Python**
- **Pandas**
- **NumPy**
- **Scikit-learn**
- **Matplotlib**
- **PE/ELF analysis tools**
- **Static malware-analysis datasets**
- **Jupyter Notebook**

The analysis should be conducted in a controlled environment and should avoid executing unknown malicious binaries.

Performance Evaluation

The model can be evaluated using:

Accuracy

Measures the overall percentage of correctly classified samples.

Precision

Measures how many samples predicted as malicious are actually malicious.

Recall

Measures how many actual malicious samples were successfully detected.

F1-Score

Provides a balance between precision and recall.

Confusion Matrix

Shows:

- True Positive
- True Negative
- False Positive
- False Negative

For cybersecurity applications, false negatives are particularly important because an undetected malicious sample may represent a security risk.

Analysis

Different algorithms can produce different results depending on the dataset and features. Tree-based algorithms such as Random Forest can be useful because they can handle multiple feature types and provide feature-importance information.

For example, if imported API patterns and entropy contribute strongly to classification, these features may be considered important indicators for the model.

Expected Result

AI-based malware analysis can automate the initial screening of large numbers of samples and help security researchers prioritize suspicious files for deeper investigation.

However, AI should not be treated as a final authority because malware authors can modify code, use obfuscation, or create previously unseen variants that may not match the training dataset.

Conclusion

AI and Machine Learning can significantly improve malware reverse-engineering workflows by automatically analyzing static characteristics and identifying suspicious patterns.
Classification models can reduce the amount of manual analysis required and help researchers prioritize potentially malicious samples. Nevertheless, AI-based detection has limitations such as false positives, false negatives, dataset bias, and adversarial modifications. Therefore, AI should complement rather than completely replace traditional malware analysis techniques.

---

4. Reverse Engineering of APIs

Using AI Introduction

Application Programming Interfaces (APIs) allow different software components to communicate with each other. When API documentation is incomplete, outdated, or unavailable, understanding the API can become difficult. AI can assist in reverse engineering APIs by analyzing request-response pairs, source code, endpoint structures, parameters, and observed behavior.

This activity can be safely performed using a **documented/open API or a controlled test API** rather than attempting to access unauthorized systems.

Problem Identification

The problem is to determine whether AI can infer the functionality and structure of an API from its observable behavior and available technical information. The objective is to identify endpoints, parameters, request methods, response formats, and relationships between API operations and generate documentation from these observations.

API Information to Analyze

The following information can be collected:

- API endpoints
- HTTP methods
- Request parameters
- Headers
- Authentication requirements
- Request body format
- Response format
- HTTP status codes
- Error messages
- JSON/XML structures
- Relationships between endpoints

For example:

GET /users

may return user information, while:

POST /users

may create a new user.

AI can analyze these patterns and infer the purpose of different operations.

Tools Used

- **Postman** – API request and response testing
- **Swagger/OpenAPI** – API specification comparison
- **Python** – data processing

- **VS Code** – source-code analysis
- **JSON tools** – response analysis
- **AI/LLM tools** – API explanation and documentation generation
- **Browser Developer Tools** – observing controlled API communication

Methodology

The workflow is:

API Selection → Request Generation → Response Collection → Structure Analysis → AI-Based Interpretation → Documentation Generation → Specification Comparison → Accuracy Evaluation

First, a documented or controlled API is selected. Different requests are sent using appropriate HTTP methods.

The responses are recorded and analyzed for:

- Data fields
- Data types
- Status codes
- Error conditions
- Relationships between requests and responses

The collected information is then provided to an AI system for interpretation.

AI-Based Inference

AI can analyze API interactions and generate descriptions such as:

- Purpose of an endpoint
- Required parameters
- Optional parameters
- Expected response fields
- Possible error conditions
- Relationship between endpoints

For example, if an endpoint accepts a user ID and returns user details in JSON format, AI can infer that the endpoint is likely responsible for retrieving information about a specific user.

Documentation Generation

AI can convert the observed API behavior into structured documentation containing:

Component	Description
Endpoint	URL path used by the API
Method	GET, POST, PUT, DELETE, etc.
Parameters	Required and optional inputs
Request Body	Data sent to the server
Response	Data returned by the API
Status Codes	Success and error responses
Function	Purpose of the endpoint

The generated documentation can then be compared with the actual OpenAPI/Swagger specification.

Evaluation

The quality of AI-generated documentation can be evaluated based on:

- Endpoint identification accuracy
- Parameter identification accuracy
- Correctness of HTTP methods
- Response-field accuracy
- Error-condition identification
- Completeness of documentation

Any mismatch between the AI-generated documentation and the actual API specification should be recorded and analyzed.

Limitations

AI-based API reverse engineering has several limitations:

- Hidden endpoints may not be discovered.
- Authentication may prevent access to some operations.
- Similar endpoints may be confused.
- Dynamic parameters may be difficult to infer.
- AI may incorrectly interpret an endpoint's purpose.
- Observed behavior may not represent every possible API state.

Therefore, AI-generated documentation must be verified against the actual API behavior or official specification whenever available.

Expected Result

AI can effectively assist in recovering API documentation from observable request-response behavior. It can reduce the time required to understand unfamiliar APIs and generate structured technical documentation.

Conclusion

AI provides a useful approach for API reverse engineering and documentation recovery. By analyzing requests, responses, source code, and API specifications, AI can infer endpoints, parameters, data formats, and functionality. However, AI-generated documentation should always be validated against the actual API because observed behavior may be incomplete and AI may make incorrect assumptions. The combination of **AI analysis, controlled testing, and human verification** provides a reliable approach to API understanding.

Overall Comparison

Topic	Main AI/ML Technique	Major Application	Evaluation
Machine Learning for Reverse Engineering	Classification & Clustering	Code classification and similarity detection	Accuracy, F1-score, Silhouette Score
AI for Source Code Analysis	NLP & LLMs	Code explanation and program understanding	Human verification and explanation accuracy
AI-Based Malware Analysis	ML Classification	Suspicious/malicious pattern detection	Precision, Recall, F1-score, Confusion Matrix
AI for API Reverse Engineering	LLM-based inference	Endpoint and documentation recovery	Endpoint, parameter and documentation accuracy